Volunteer Induction

Problem Statements — Software Division
Programming Club, IIT Indore

Welcome

The Software Division of the Programming Club, IIT Indore is looking for new volunteers. To apply, build a project from the problem statements in this document and submit it. There are six problem statements across three difficulty levels. Pick whichever matches your current skill level and interest. This is your chance to show how you think, build and learn.

Problem Statements at a Glance

Level	Problem Statement	What you will practise
Beginner	1. Dice Roll Website	HTML, CSS, JavaScript, DOM, basic logic
Beginner	2. Quiz Platform	State handling, data structures, UI flow
Intermediate	3. Protein Intake Tracker	Data persistence, forms, charts, calculations
Intermediate	4. Task Management Dashboard	CRUD, filtering, state management, UX
Advanced	5. Portfolio Site Builder	Templating, dynamic rendering, export/deploy
Advanced	6. Developer Community Platform	Full stack, auth, databases, APIs

Already built something? Bring it!

APPLICANTS CAN ALSO COME AND SHOW US ANY PROJECT THEY HAVE BUILT BEFOREHAND. You do not have to build one of the listed problem statements. If you already have a project you are proud of, present it to us instead.

The listed problems are only ideas

The problem statements below are **suggestions, not a rule**. There is no requirement to pick one of them. You are free to build any website of your own idea, as long as it follows the general rules below.

General Rules (apply to every problem statement)

- **Must be a website.** Every project is to be built as a website (frontend only, or full stack for the harder ones).
- **GitHub repository is mandatory.** Push your code to a public GitHub repository with regular, meaningful commits (not one single commit at the end). Include a good README: project description, features, tech stack, setup/run instructions and screenshots.
- **Hosting is optional but recommended.** A live link (GitHub Pages, Netlify, Vercel, Render, etc.) makes your submission much easier to evaluate and leaves a strong impression.
- **Use of AI is permitted, but you must understand your code.** You should be able to explain what code you have written, how it works, and why you made your design choices. Expect questions about your code in the interview; code you cannot explain will count against you.
- **Originality.** Do not copy another applicant's project. Using libraries, tutorials and documentation is fine, but the work submitted must be your own.
- **Go beyond the minimum.** Each statement lists core requirements and optional stretch goals. Completing the core cleanly is better than half-finishing many extras.

BEGINNER LEVEL

Problem Statement 1: Dice Roll Website

Objective

Build an interactive website that simulates rolling one or more dice, with a polished and responsive interface. This is a first step to show you can connect HTML structure, CSS styling and JavaScript behaviour.

Description

The website simulates rolling dice. The user clicks a button and gets a random result, which is displayed visually (dice face or pips), not just as a number.

It should be fun to use: animations, sounds and a clean design all help make the experience feel like a real roll.

Core Requirements (must have)

- Roll a standard six-sided die using a button click, with a randomly generated result between 1 and 6.
- Display the dice face visually (drawn with CSS, SVG or images) and update it on every roll.
- Show a roll animation (shake, spin, or cycling faces) before the final result appears.
- Allow rolling **two dice** together and show the total.
- Keep a **roll history** of the last several results on the page.
- Responsive layout that works on both mobile and desktop.

Stretch Goals (optional)

- Choose the number of dice (1 to 6) and different dice types (d4, d8, d12, d20).
- Sound effects with a mute toggle.
- Statistics panel: count of each outcome, average roll, a simple bar chart of frequencies.
- A two-player mode with scores and turns.
- Dark/light theme toggle; remember history using localStorage.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- A short note in the README on how the randomness is generated.

Suggested Tech

HTML, CSS and vanilla JavaScript are enough. You may use a framework (React, Vue, etc.) if you are comfortable with it.

Evaluation Focus

- Correct and unbiased random logic.
- Visual quality of the dice and animation.
- Clean separation of HTML, CSS and JS; responsive design.
- Ability to explain how your code works.

BEGINNER LEVEL

Problem Statement 2: Quiz Platform

Objective

Build a web-based quiz platform where users can attempt multiple-choice quizzes, get scored, and review their answers. This tests your handling of data, application state and multi-screen UI flow.

Description

The platform presents questions one at a time (or all at once), records the user's choices, calculates a score and shows a results screen. Quiz content is stored in a structured format such as a JSON file, so adding a new quiz does not require changing the code.

Core Requirements (must have)

- At least **two quizzes** on different topics (for example: programming basics, general knowledge), each with 10 or more questions.
- Start screen to choose a quiz, then a question screen with multiple-choice options.
- Progress indicator (for example Question 4 of 10) and Next/Previous navigation.
- Final result screen showing the score, percentage and a review of correct vs. incorrect answers.
- Questions stored in JSON or a JS data structure, kept separate from the UI logic.
- Option to restart or choose another quiz; responsive layout.

Stretch Goals (optional)

- Countdown timer per question or per quiz, with auto-submit.
- Randomise the order of questions and options.
- Leaderboard stored in localStorage (or a backend).
- Difficulty levels, categories and negative marking.
- Admin-style page to create new quizzes through a form.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- At least two sample quizzes included in the repository.

Suggested Tech

HTML, CSS and JavaScript. Optionally a framework, or a simple backend (Node/Express, Flask) if you want to store scores.

Evaluation Focus

- Correct scoring and state handling.
- Clean data structure for questions.
- Smoothness of the user flow and UI.
- Ability to explain your code and design choices.

INTERMEDIATE LEVEL

Problem Statement 3: Protein Intake Tracker

Objective

Build a website that helps a user track daily protein consumption against a personalised goal. The focus is on forms, calculations, data persistence and presenting data clearly.

Description

A user sets up a profile (weight, activity level, goal) and the app suggests a daily protein target. The user then logs the food they eat through the day and sees how close they are to the target, with history over time.

Core Requirements (must have)

- Profile/goal setup: the user enters body weight and activity level, and the app computes a recommended daily protein target (for example grams per kg of body weight). The user should also be able to set the target manually.
- Add, edit and delete food entries with name, quantity and protein in grams.
- A built-in list of common foods (eggs, paneer, dal, chicken, milk, etc.) with protein per serving to make logging fast.
- Daily summary with a progress bar showing consumed vs. target and remaining protein.
- History view: see previous days, with a weekly chart of protein intake.
- Data persists after refresh (localStorage, or a database if you build a backend).

Stretch Goals (optional)

- User accounts with login so data is stored per user (requires a backend).
- Search a food nutrition API to fetch protein values automatically.
- Track calories, carbs and fats along with protein.
- Reminders and streaks (days the goal was met); export history as CSV.
- Dark mode and fully mobile-friendly design.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- A short explanation in the README of how the daily target is calculated and which data source was used for food values.

Suggested Tech

A frontend framework such as React or Vue is recommended, with a charting library (Chart.js, Recharts). A backend (Node, Flask, Django) and database (SQLite, MongoDB) are optional unless you attempt accounts.

Evaluation Focus

- Correct calculations and sensible validation of inputs.
- Quality of the data model and persistence.
- Usefulness and clarity of charts and summaries.
- Code structure and ability to explain it.

Note: this is a tracking tool for learning purposes. It does not need to give medical advice.

INTERMEDIATE LEVEL

Problem Statement 4: Task Management Dashboard

Objective

Build a dashboard for managing tasks and projects, similar in spirit to Trello or Todoist. This tests state management, CRUD operations, filtering and thoughtful UX.

Description

Users create tasks, organise them by status, priority and due date, and get an at-a-glance overview of their workload through a dashboard. The experience should feel fast and intuitive.

Core Requirements (must have)

- Create, read, update and delete tasks with title, description, priority (low/medium/high) and due date.
- Task status columns such as **To Do, In Progress, Done** (Kanban board), with the ability to move tasks between them (drag-and-drop or buttons).
- Filter and search tasks by status, priority, due date or keyword; sort by due date or priority.
- Dashboard summary: total tasks, completed, pending and overdue; a simple chart of tasks by status or priority.
- Visual highlighting of overdue and high-priority tasks.
- Data persists after refresh; responsive design.

Stretch Goals (optional)

- User authentication and per-user data via a backend with a database.
- Multiple projects/boards, labels or tags, and sub-tasks with checklists.
- Drag-and-drop reordering within columns.
- Team features: assign tasks to other users, comments, activity log.
- Calendar view, notifications for due dates, dark mode.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- Screenshots or a short demo GIF of the dashboard in the README.

Suggested Tech

React (or Vue/Svelte) is recommended for managing state. Optional backend: Node/Express, Django or Flask with a database such as MongoDB, PostgreSQL or SQLite.

Evaluation Focus

- Complete and bug-free CRUD and filtering.
- State management approach and component structure.
- UX quality: responsiveness, clarity, interactions.
- Ability to explain your code and design decisions.

ADVANCED LEVEL

Problem Statement 5: Portfolio Site Builder

Objective

Build a web application that lets a person create their own personal portfolio website without writing code: they fill in forms, choose a theme, preview the result live and then publish or export it.

Description

Think of a tool where a student enters their details (about, skills, projects, education, links) and the system generates a polished, ready-to-share portfolio. The challenge is in designing a flexible data model, a theming/templating system and a smooth live-editing experience.

Core Requirements (must have)

- A guided editor with forms for sections: personal info, about, skills, projects (title, description, links, images), experience/education and contact details.
- **Live preview** that updates instantly as the user edits.
- At least **three distinct templates/themes**, switchable without losing content; customisable accent colour or fonts.
- Ability to add, remove and reorder sections and projects.
- **Export** the finished portfolio as a standalone HTML/CSS bundle (zip) that can be hosted anywhere, or generate a shareable public link.
- Saved work: the user's data persists and can be edited later; generated portfolio is responsive and accessible.

Stretch Goals (optional)

- User accounts so each user can manage a portfolio, with a unique public URL such as /username.
- Image upload, drag-and-drop section ordering and a built-in markdown editor for project descriptions.
- One-click deployment to GitHub Pages or Netlify.
- Import data from a GitHub profile (repos, languages) or a resume.
- SEO fields (title, description, social preview), analytics for portfolio visits, custom domain instructions.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- A sample generated portfolio (exported or hosted) included as a demo.
- A short design write-up in the README: data model, how templates work, how export is generated.

Suggested Tech

React/Next.js (or any framework) on the frontend; Node/Express, Django or Flask with a database if you add accounts and publishing. Templating can be done with component-based themes or a template engine.

Evaluation Focus

- Architecture: data model and templating design.
- Quality and polish of the live preview and generated sites.
- Completeness of the export/publish flow.
- Code quality, documentation and ability to explain the design.

ADVANCED LEVEL

Problem Statement 6: Developer Community Platform

Objective

Build a full-stack platform where developers can connect, ask questions, share knowledge and discuss ideas, in the spirit of Stack Overflow, Dev.to or a Discord-like community. This tests authentication, databases, API design and scalability thinking.

Description

The platform allows people to sign up, create posts or questions, reply, vote and follow topics. A good solution combines a clean user experience with a well-structured backend and sensible moderation basics.

Core Requirements (must have)

- User registration, login and logout with secure password handling and sessions or tokens (JWT).
- User profile pages with bio, skills and activity.
- Create, edit and delete **posts/questions** with tags and markdown/code formatting.
- Comments or answers on posts, with the ability to upvote/downvote and mark an accepted answer.
- Feed with search, tag filtering and sorting (newest, top).
- A properly designed **backend API and database** (users, posts, comments, votes, tags) with input validation and access control (users can edit only their own content).
- Responsive UI and basic error handling.

Stretch Goals (optional)

- Real-time features: live notifications or chat using WebSockets.
- Follow users and tags; personalised feed; bookmarks.
- Reputation points and badges.
- Roles and moderation (admin can remove content, report system).
- Pagination or infinite scroll, rate limiting, email verification, OAuth login (GitHub/Google).
- Deployed with a hosted database and a CI pipeline.

Deliverables

- A **public GitHub repository** with the complete source code and a clear commit history.
- A **README.md** covering overview, features, tech stack, setup steps and screenshots.
- **Live hosted link** (optional but recommended) added to the README.
- Documentation of the API endpoints and a database schema diagram or description in the README.
- Instructions for running the project locally, including environment variables (never commit secrets).

Suggested Tech

Frontend: React/Next.js or similar. Backend: Node/Express, Django, Flask or FastAPI. Database: PostgreSQL, MySQL or MongoDB. Hosting on Render, Railway, Vercel or similar.

Evaluation Focus

- Backend design: schema, API structure, authentication and security.
- Completeness of core community features.
- Code organisation, documentation and testing effort.
- Ability to explain architecture and trade-offs.

Submission Checklist

- Project is a **website** (chosen problem statement, your own idea, or an earlier project you have built).
- Code is pushed to a **public GitHub repository** with a proper README.
- Live link included in the README if you have hosted it (optional but recommended).
- You can **explain every part of the code you submit**, including any AI-assisted portions.
- You are ready to demo your project and answer questions about it.

What We Will Look At

Functionality (does it work without bugs), **code quality** (readable, organised), **UI/UX** (clean, responsive), **understanding** (can you explain and modify your own code) and **effort and initiative** (extra features, documentation, hosting).

Reminder

Applicants can also come and show us any project they have built beforehand. The listed problem statements are only ideas; you are not bound to pick any of them.

We look forward to seeing what you build. Good luck!

Programming Club, IIT Indore — Software Division